Traffic Light Control System - FSM Design

T-Junction (Signals A, B, C) | Moore FSM | Demand-driven with fixed-order fallback | Verilog/RTL reference

1. Observations from the specification

- **Geometry:** A = West arm (W to E), B = East arm (E to W), C = South arm (S to N). C has no straight-through movement.
- **Phases:** P0 all-red, P1 A_G1, P2 B_G1, P3 AB_G1, P4 C_GREEN, P5 A_G2, P6 B_G2, and P7 amber transition. Every phase change must pass through all-red (P0).
- **Safety rule:** only the listed compatibility-matrix combinations are legal; anything else is illegal and must force P0.
- **Outputs:** per signal exactly one of R / Y / G1 / G2 / G (C uses G only). In Green-2 all other signals are red.

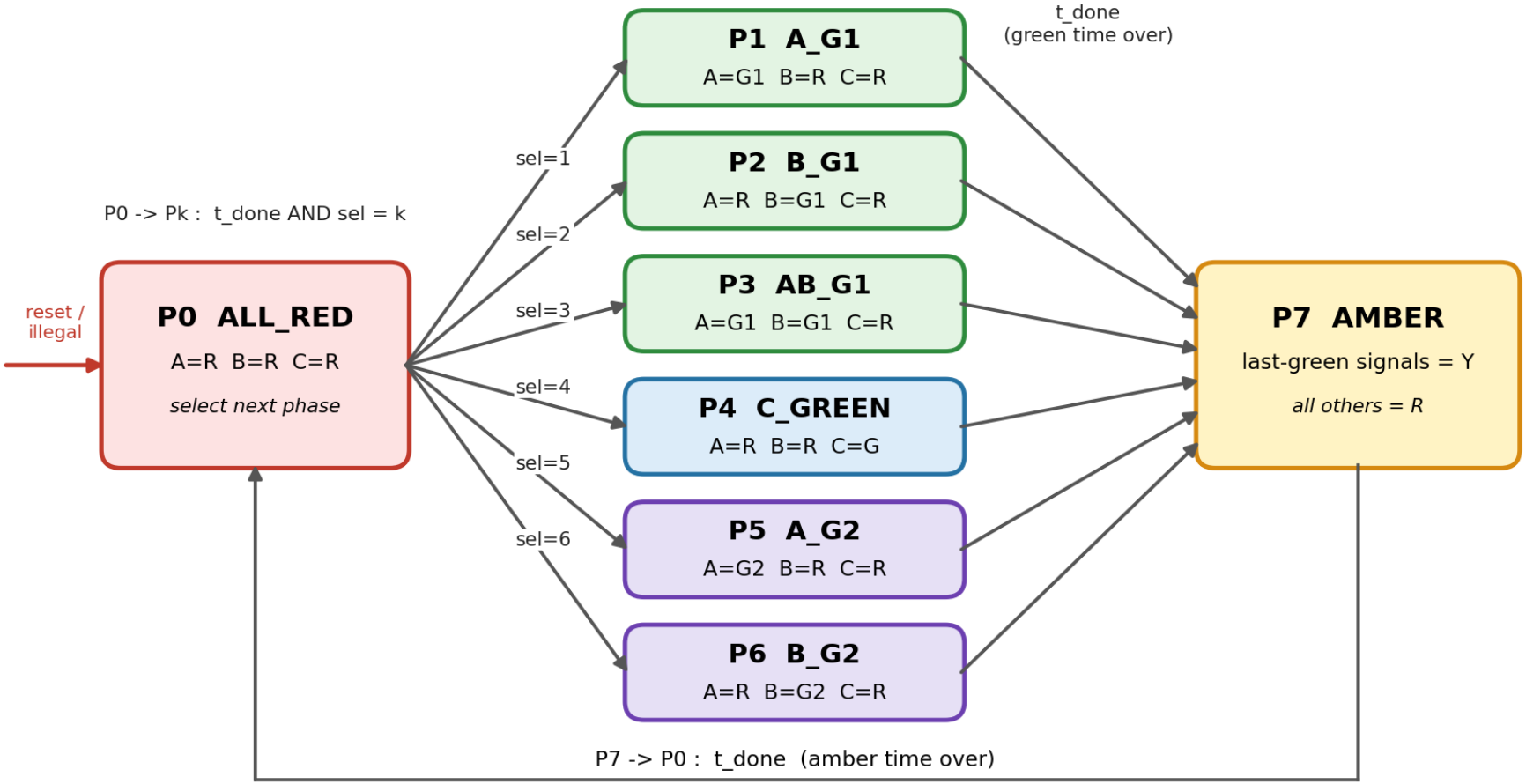
2. Gaps found in the spec and how this FSM resolves them

#	Gap / ambiguity in spec v5	Resolution used in this design
1	P7 (amber) is defined but missing from the Section 6 sequence.	Sequence becomes Pk -> P7 -> P0 -> next phase. Single P7 state; the signals that were green in the last phase show Y, all others R.
2	Compatibility matrix has no amber (Y) rows, so amber would be 'illegal' by the unlisted = illegal rule.	Four transition-only rows added (T1-T4: Y R R, R Y R, Y Y R, R R Y). They occur only in P7 and never overlap any green.
3	Demand inputs are mentioned but not defined.	Five request inputs (reqA, reqB, reqC, reqA_turn, reqB_turn) mapped to six phase requests (Section 5).
4	No timing values.	Parameterised timers; suggested defaults are my assumption and can be changed freely.
5	Free-left is 'permissive if enabled' with no enable.	Input yield_en gates the informational outputs C_free_left (P4) and B_free_left (P6). No lamp output changes.
6	Illegal-state recovery required.	4-bit state register; codes 1000-1111 are unused and force P0 next clock. The 'last' register is also sanitised.
7	'Fixed-order fallback' when no request is pending.	Next phase = the one following the last served phase in P1,P2,...,P6,P1 order, so the Section 6 cycle is preserved.

3. Design decisions (confirmed by you)

- **Amber:** one shared P7 state with a 'last' register remembering which phase was green.
- **Next phase:** demand-driven round-robin scan starting after the last served phase; if no request, fixed order.
- **Style:** Moore machine - lamp outputs depend only on state (and 'last' inside P7).

4. FSM state diagram



Self-loops: P0, P1-P6 and P7 each stay in the same state while **t_done = 0** (timer running). **sel** is the phase chosen by the demand scan (Section 5). Reset, and any illegal state code (1000-1111), go to P0.

5. State table, outputs and demand selection

5.1 State table and lamp outputs (Moore)

State	Name	Code	A	B	C	Permitted movements	Duration
P0	ALL_RED	0000	R	R	R	Clearance, no protected movement	T_CLR
P1	A_G1	0001	G1	R	R	A straight to East	T_G1
P2	B_G1	0010	R	G1	R	B straight to West	T_G1
P3	AB_G1	0011	G1	G1	R	A straight + B straight (parallel)	T_G1
P4	C_GREEN	0100	R	R	G	C right to East; C left if yield_en	T_GC
P5	A_G2	0101	G2	R	R	A straight + right to C + U-turn	T_G2
P6	B_G2	0110	R	G2	R	B straight + U-turn; B left if yield_en	T_G2
P7	AMBER	0111	Y / R*	Y / R*	Y / R*	Transition after any green phase	T_AMB

* P7: A = Y if last is P1, P3 or P5; B = Y if last is P2, P3 or P6; C = Y if last is P4; otherwise R. Suggested defaults (assumption, in seconds/ticks): T_G1 = 10, T_G2 = 12, T_GC = 10, T_AMB = 3, T_CLR = 2.

5.2 Inputs and outputs

Signal	Dir	Description
clk, rst_n	in	Clock; active-low asynchronous reset (reset goes to P0, last = P6).
reqA, reqB, reqC	in	Vehicle waiting on A straight / B straight / C.
reqA_turn, reqB_turn	in	Vehicle waiting for the A turn/U-turn phase (P5) / B turn/U-turn phase (P6).
yield_en	in	Enables the free-left flags (static configuration).
A_R,A_Y,A_G1,A_G2 B_R,B_Y,B_G1,B_G2 C_R,C_Y,C_G	out	Lamp outputs of signals A, B and C (one-hot per signal).
B_free_left, C_free_left	out	Yield-controlled left flags, active only in P6 / P4 when yield_en = 1.

5.3 Demand to phase-request mapping

Phase request	Logic	Meaning
rq1 (P1)	reqA AND NOT reqB	Only A straight is waiting
rq2 (P2)	reqB AND NOT reqA	Only B straight is waiting
rq3 (P3)	reqA AND reqB	Both straight movements waiting, serve in parallel
rq4 (P4)	reqC	C is waiting
rq5 (P5)	reqA_turn	A turn/U-turn waiting
rq6 (P6)	reqB_turn	B turn/U-turn waiting

5.4 Next-phase selection at the end of P0 (round-robin, with fixed-order fallback)

last served	Scan order (first active request wins)	If no request is active
P1	P2, P3, P4, P5, P6, P1	P2
P2	P3, P4, P5, P6, P1, P2	P3
P3	P4, P5, P6, P1, P2, P3	P4
P4	P5, P6, P1, P2, P3, P4	P5
P5	P6, P1, P2, P3, P4, P5	P6
P6 (also reset value)	P1, P2, P3, P4, P5, P6	P1

6. Transition table

Current	Condition	Next	Register action
any (reset)	rst_n = 0	P0	last = P6, timer = T_CLR-1
P0	t_done = 0	P0	timer counts down
P0	t_done AND sel = 1	P1	last = P1, load T_G1
P0	t_done AND sel = 2	P2	last = P2, load T_G1
P0	t_done AND sel = 3	P3	last = P3, load T_G1
P0	t_done AND sel = 4	P4	last = P4, load T_GC
P0	t_done AND sel = 5	P5	last = P5, load T_G2
P0	t_done AND sel = 6	P6	last = P6, load T_G2
P1 to P6 (each)	t_done = 0	same state	timer counts down
P1 to P6 (each)	t_done = 1	P7	load T_AMB; last is kept
P7	t_done = 0	P7	timer counts down
P7	t_done = 1	P0	load T_CLR
1000 to 1111 (illegal)	always	P0	load T_CLR; all lamps red

Every green phase is followed by P7 then P0, so a green can never change directly to another green. **sel** comes from Section 5.4 and is only evaluated in P0.

7. Safety check: legal lamp combinations

7.1 Every state against the compatibility matrix

State	A	B	C	Matrix row	Allowed
P0	R	R	R	R R R	YES
P1	G1	R	R	G1 R R	YES
P2	R	G1	R	R G1 R	YES
P3	G1	G1	R	G1 G1 R	YES
P4	R	R	G	R R G	YES
P5	G2	R	R	G2 R R	YES
P6	R	G2	R	R G2 R	YES
P7 (last = P1/P5)	Y	R	R	T1 (added)	YES, transition only
P7 (last = P2/P6)	R	Y	R	T2 (added)	YES, transition only
P7 (last = P3)	Y	Y	R	T3 (added)	YES, transition only
P7 (last = P4)	R	R	Y	T4 (added)	YES, transition only

7.2 Prohibited rows and why the FSM cannot produce them

Prohibited (A B C)	Why unreachable
G1 R G, R G1 G, G1 G1 G	C = G exists only in P4, where A = B = R.
G2 G1 R, G1 G2 R, G2 G2 R	G2 exists only in P5 (B = R) and P6 (A = R); P3 is the only state with two greens and both are G1.
G2 R G, R G2 G, G2 G2 G	G2 and C = G never share a state.

8. Verification checklist (spec Section 8)

ID	Requirement	How to check	Pass condition
V1	No prohibited matrix row generated	Assertion on {A,B,C} lamp state every clock	Tuple is in the legal set (7.1) on every cycle
V2	Amber and all-red between phases	Monitor state sequence	Any green Pk is followed by P7 then P0 then next; never Pk to Pj directly
V3	Reset goes to P0	Assert rst_n low, then check	state = P0 and all lamps red
V4	Illegal-state recovery	Force state to 1000..1111 (testbench force/release)	Next clock: P0, all red
V5	A_G1 and B_G1 together only in P3	Assert (A_G1 AND B_G1) implies state = P3	Never violated
V6	C_GREEN never overlaps other greens	Assert C_G implies A_R AND B_R	Never violated
V7	Green-2 exclusive	Assert (A_G2 OR B_G2) implies other signals red	Never violated
V8	One active lamp per signal	Assert onehot(A_R,A_Y,A_G1,A_G2) etc.	True every cycle
V9	P0 all-red	Assert state = P0 implies all R	True every cycle
V10	Waveform inspection	GTKWave: clk, rst_n, req*, state, last, cnt, t_done, all lamps	Sequence matches the diagram; timings match parameters

9. Verilog-ready encoding

9.1 State encoding (4 bit)

P0	P1	P2	P3	P4	P5	P6	P7	1000-1111
0000	0001	0010	0011	0100	0101	0110	0111	illegal, go to P0

9.2 Lamp output vector {A_R,A_Y,A_G1,A_G2, B_R,B_Y,B_G1,B_G2, C_R,C_Y,C_G}

State	Vector
P0	1000 1000 100
P1	0010 1000 100
P2	1000 0010 100
P3	0010 0010 100
P4	1000 1000 001
P5	0001 1000 100
P6	1000 0001 100
P7, last = P1/P5	0100 1000 100
P7, last = P2/P6	1000 0100 100
P7, last = P3	0100 0100 100
P7, last = P4	1000 1000 010

9.3 Reference RTL (Moore FSM, 3-process style)

```
module traffic_fsm #(parameter T_G1=10, T_G2=12, T_GC=10, T_AMB=3, T_CLR=2)(
  input  wire clk, rst_n,
  input  wire reqA, reqB, reqC, reqA_turn, reqB_turn, yield_en,
  output reg  A_R, A_Y, A_G1, A_G2, B_R, B_Y, B_G1, B_G2, C_R, C_Y, C_G,
  output reg  B_free_left, C_free_left);

  localparam [3:0] P0=0, P1=1, P2=2, P3=3, P4=4, P5=5, P6=6, P7=7;
  reg [3:0] state, next_state;
  reg [2:0] last;            // last served green phase (1..6)
  reg [7:0] cnt;             // down-counter, t_done when 0
  wire      t_done = (cnt == 0);
  wire [6:1] rq = {reqB_turn, reqA_turn, reqC, reqA & reqB, reqB & ~reqA, reqA & ~reqB};

  function [7:0] dur(input [3:0] s);
    case (s) P0: dur=T_CLR; P1,P2,P3: dur=T_G1; P4: dur=T_GC;
             P5,P6: dur=T_G2; P7: dur=T_AMB; default: dur=T_CLR; endcase
  endfunction

  function [2:0] pick(input [2:0] l, input [6:1] r);   // round-robin scan
    integer i; reg [3:0] c; reg [2:0] ls; reg found;
    begin
      ls = (l==0 || l>6) ? 3'd6 : l;
      found = 0; pick = (ls==6) ? 3'd1 : ls + 3'd1;      // fixed-order fallback
      for (i=1; i<=6; i=i+1) begin
        c = ls + i; if (c > 6) c = c - 6;
        if (!found && r[c]) begin pick = c[2:0]; found = 1; end
      end
    end
  endfunction

  // 1) state register
  always @(posedge clk or negedge rst_n)
    if (!rst_n) begin state<=P0; last<=3'd6; cnt<=T_CLR-1; end
    else begin
      state <= next_state;
      if (next_state != state) begin
        cnt <= dur(next_state) - 1;
        if (state==P0 && next_state>=P1 && next_state<=P6) last <= next_state[2:0];
      end else if (cnt != 0) cnt <= cnt - 1;
      if (last==0 || last>6) last <= 3'd6;
    end

  // 2) next-state logic (illegal codes fall to P0)
  always @* begin
    case (state)
      P0:                         next_state = t_done ? {1'b0, pick(last, rq)} : P0;
      P1,P2,P3,P4,P5,P6:          next_state = t_done ? P7 : state;
      P7:                         next_state = t_done ? P0 : P7;
      default:                    next_state = P0;
    endcase
  end

  // 3) Moore outputs
  always @* begin
    A_Y=0; A_G1=0; A_G2=0; B_Y=0; B_G1=0; B_G2=0; C_Y=0; C_G=0;
    B_free_left=0; C_free_left=0;
    case (state)
      P1: A_G1 = 1;
      P2: B_G1 = 1;
      P3: begin A_G1 = 1; B_G1 = 1; end
      P4: begin C_G = 1; C_free_left = yield_en; end
      P5: A_G2 = 1;
      P6: begin B_G2 = 1; B_free_left = yield_en; end
      P7: begin
            A_Y = (last==1) || (last==3) || (last==5);
            B_Y = (last==2) || (last==3) || (last==6);
            C_Y = (last==4);
          end
      default: ;                    // P0 and illegal codes: all red
    endcase
    A_R = ~(A_Y | A_G1 | A_G2);
    B_R = ~(B_Y | B_G1 | B_G2);
    C_R = ~(C_Y | C_G);
  end
endmodule
```

Note: the RTL was reviewed by hand and the phase logic (lamp tuples, P7 rule, round-robin selection, Pk-P7-P0 sequencing) was checked with a Python model. It has not been compiled in a Verilog simulator here, so please run it in your simulator (Icarus/ModelSim) with the V1-V10 checks before submission.